

```

krishna $ packer build .
amazon-eks.gmi: output will be in this color.

==> amazon-eks.gmi: Prevalidating any provided VPC information
==> amazon-eks.gmi: Prevalidating AMI Name: glarimy-ami-1683011997
amazon-eks.gmi: Found Image ID: ami-007855ac798b5175e
==> amazon-eks.gmi: Creating temporary keypair: packer_6450b99d-6d6e-2f2a-eed4-942
==> amazon-eks.gmi: Creating temporary security group for this instance: packer_64
==> amazon-eks.gmi: Authorizing access to port 22 from [0.0.0.0/0] in the temporar
==> amazon-eks.gmi: Launching a source AWS instance...
amazon-eks.gmi: Instance ID: i-0f6f68bc8ab42cd69
==> amazon-eks.gmi: Waiting for instance (i-0f6f68bc8ab42cd69) to become ready...
==> amazon-eks.gmi: Using SSH communicator to connect: 3.86.224.19
==> amazon-eks.gmi: Waiting for SSH to become available...
==> amazon-eks.gmi: Connected to SSH!
==> amazon-eks.gmi: Stopping the source instance...
amazon-eks.gmi: Stopping instance
==> amazon-eks.gmi: Waiting for the instance to stop...
==> amazon-eks.gmi: Creating AMI glarimy-ami-1683011997 from instance i-0f6f68bc8a
amazon-eks.gmi: AMI: ami-0033e806df9f271e4
==> amazon-eks.gmi: Waiting for AMI to become ready...
==> amazon-eks.gmi: Skipping Enable AMI deprecation...
==> amazon-eks.gmi: Terminating the source AWS instance...
==> amazon-eks.gmi: Cleaning up any extra volumes...
==> amazon-eks.gmi: No volumes to clean up, skipping
==> amazon-eks.gmi: Deleting temporary security group...
==> amazon-eks.gmi: Deleting temporary keypair...
Build 'amazon-eks.gmi' finished after 3 minutes 31 seconds.

==> Wait completed after 3 minutes 31 seconds

==> Builds finished. The artifacts of successful builds are:
--> amazon-eks.gmi: AMIs were created:
us-east-1: ami-0033e806df9f271e4

```

Figure 3: Packer build output

finish as it involves interaction with AWS, spinning a new EC2, deploying the source AMI, and building the new AMI. See Figure 3 to understand the steps involved in the whole process.

In my case, a new AMI with ID `ami-058213aa644fe1b60` in the us-east-1 region is created. It will be listed under the *My AMI* tab on the AWS AMI console page.

Using variables

The Packer HCL template files can also define and use variables for reusability. For example, the `ssh_username` in the following template is referred to a variable named `var.uname`.

```

source "amazon-eks" "glarimy-ubuntu" {
  ssh_username = var.uname
  ami_name = "glarimy-ubuntu-{{timestamp}}"
  ...
}
...

```

Where is this variable defined and initialised? The variables can be defined in the same template file or in a separate file. The latter is a better approach for obvious reasons. The default name of such a file is `variables.pkr.hcl`.

Packer automatically locates this file locally.

In our case, the `variables.pkr.hcl` file has the following block:

```

variable "uname" {
  type = string
  default = "ubuntu"
}

```

As you can see, the file contains a definition for a variable named `uname`. The arguments specify that the `uname` is a string and 'ubuntu' is its default value. You can also supply validation rules and descriptions.

Once the variables are defined, you can initialise them. There are multiple ways to do this. You may use environmental variables, command line arguments, etc. In our case, we don't do anything as we are happy with the default value.

Another interesting argument in the above `source` file is the

way we name the new AMI. Instead of giving a static name like 'glarimy-ubuntu', we are using the current timestamp as the prefix. That way, every time Packer runs this template as part of the CI/CD routine, it creates a new AMI with a unique name, which we can refer to later.

Validating, formatting, and building the above template should happen smoothly. Note that the command syntax to build the image should be changed to the following, as Packer needs to locate multiple files for building.

```
packer build .
```

In the above, we replaced the template file name with a dot to tell Packer to locate all the `.pkr.hcl` files in the current folder.

Selecting base AMI dynamically

Another feature of Packer is that it can dynamically locate the source image from a named data source. For example, our templates so far specify a particular AMI with ID `ami-007855ac798b5175e` as the source. This may not always be helpful, especially in automated CI/CD processes. What if you want to pick up the latest Ubuntu AMI automatically as the source, no matter what its ID? Let's enhance our template for that purpose.